The Linear-Cost-Scaling Test as Standalone Operational Procedure in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)
ORCID: 0009-0004-8065-3235
Date: May 7, 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the linear-cost-scaling test — the operational procedure by which a CKS deployment is verified to satisfy the source paper's linear-cost commitment (Claim 4, §6) — as a standalone procedure specification, separable from the broader determinism, tool-agnosticism, and composition tests with which it shares architectural intent.

Abstract

The CKS pattern's linear-cost commitment (Claim 4, §6 of the source paper) is the architectural property that underwrites bottom-up adoptability and capacity planning at scale: of the costs a CKS system incurs, only storage and full-substrate read scale with substrate size; governance cost, cell execution cost, LLM cost per execution, conflict-handling cost, and onboarding cost do not. A separate derivation note formalizes the commitment itself (Li, 25 April 2026). This note formalizes the operational procedure by which a deployment is verified, at scale, to actually exhibit the cost profile the architecture commits to. The test is scale-testing across multiple substrate sizes, with three deliberate scoping moves: substrate cost is measured separately from LLM consultation cost; cost variance is assessed against the bounded-non-determinism categories the architecture permits at the substrate layer; and the verification target is the cost-curve shape across scale factors, not absolute cost adequacy or business cost-effectiveness. The note states what the test verifies, specifies the procedure as operational steps, names the pass and fail criteria, enumerates the anti-patterns the test detects, describes the deployment-verification points at which the test is run, and names the test's architectural limits.

1. Why the test needs to be formalized as standalone

The CKS pattern's linear-cost commitment is defended in the source paper across three subsections (§6.1, §6.2, §6.3) and formalized as a single contract in a prior derivation note. The contract names what scales with what — and, more centrally, what does not scale with substrate size. As an architectural specification this is sufficient; as an operational specification it leaves a gap. A deployment that satisfies the contract on paper may, in practice, exhibit super-linear cost growth at a

particular scale factor, conflate substrate cost with LLM consultation cost in its cost model, or hit vendor-specific scaling cliffs at substrate sizes the architectural specification does not anticipate. None of these failures contradicts the contract as written; each violates it in operation.

The remedy is to specify the test that verifies the contract's satisfaction at scale, and to do so as a procedure standalone enough to be cited, run, and argued with independently. Cost-scaling is foundational for capacity planning, performance regression detection, and the economic viability of large-scale CKS deployments — concerns that, in the 2024–2026 landscape, dominate the discourse around any “scalable AI” claim. A deployment that cannot show its cost-scaling profile across multiple scale factors has no defensible response to the standard reviewer question of whether the linear-cost commitment is empirical or aspirational. Naming the test as standalone gives operators, auditors, and downstream consumers a single reference for the verification, without entangling the verdict with the verdicts of other architectural tests in the same lifecycle.

This note continues the substrate-operational-properties cluster initiated by the prior tool-agnosticism-migration test note. It is the twelfth of approximately sixteen Phase A5 notes; subsequent notes formalize the conflict-coexistence test, the composition-requirements tests, and the remaining substrate-operational tests.

2. The architectural commitment under test

The test verifies the linear-cost commitment (Claim 4, §6 of the source paper), as decomposed into the operational variants the prior derivation work specifies. Restated for the operational-test context: substrate operations scale linearly with substrate size, in the sense that cost per substrate operation either remains constant across substrate sizes or scales proportionally with operation type, not super-linearly with substrate size. Cost variance across measurements at a given substrate size is bounded to the non-determinism categories the architecture permits at the substrate layer. LLM consultation cost is bounded within cells and is *not* substrate cost; the test specifically excludes LLM consultation cost from the substrate-scaling target. Cost-scaling characteristics are testable across substrate sizes, which is what makes the commitment operational rather than only architectural.

The test does not verify other architectural commitments. It does not verify tool-agnosticism preservation across migration, which is the subject of a separate test in the same Phase A5 cluster. It does not verify substrate determinism beyond the cost-variance component bounded to permitted non-determinism categories. It does not verify LLM operational cost — bounded within cells, the LLM cost is part of cell execution rather than substrate operation, and is governed by separate commitments. The narrow scoping is deliberate: a test that bundles cost-scaling with adjacent architectural concerns is harder to interpret when it fails, because the failure attributes ambiguously across the bundled concerns.

3. The substrate-cost-versus-LLM-cost distinction

The most important framing decision in the test is the distinction between substrate cost and LLM consultation cost. Both are real costs in a CKS deployment; only the former is what the linear-cost

commitment binds.

Substrate cost is the cost of substrate operations — read, write, query, and provenance recording over substrate content — and the linear-cost commitment binds it to scale linearly with substrate size, in the sense developed in §2. **LLM consultation cost** is the cost of LLM operations within cells — token cost for input, token cost for output, and any vendor-specific overhead — bounded within cells per the AI-as-substrate-mediator commitment. LLM cost may exhibit non-deterministic and non-linear behavior at the cell layer; it is *not* part of the substrate-cost-scaling target.

The distinction matters because LLM cost is inherently super-linear in some dimensions — most notably, the quadratic attention cost of the LLM context window with respect to context length. A cost model that conflates substrate cost with LLM cost would observe super-linear growth at scale and conclude that the linear-cost commitment is violated, when what is actually happening is that LLM cost at the cell layer is exhibiting expected non-linear behavior, bounded within the cell, while substrate cost continues to scale linearly. The test holds the two costs separate from the start, measures each independently, and applies the linear-cost criterion only to the substrate-cost component.

This framing identifies the canonical anti-pattern the test detects most centrally: a system that uses pure context-window memory as substrate. In such a system, the "substrate" *is* the LLM context window; reads and writes are context manipulations; "scaling the substrate" means feeding more content into the LLM context, which incurs the quadratic attention cost of context-window memory. Such a system cannot pass the test because its substrate cost is its LLM cost, and that cost grows super-linearly with content. The test exposes the conflation by separating the two costs and observing that what the system calls substrate cost is the cost of LLM context manipulation, not of operations on a persistent substrate at all.

4. The test procedure as operational steps

The procedure is scale-testing across multiple substrate sizes. The steps are operational:

Step 1 — Define the measurement protocol. Identify the substrate operations whose cost will be measured: read, write, query, provenance recording. For each, define the unit of measurement (per-operation cost in the deployment's relevant units — wall-clock time, infrastructure cost, or both — applied consistently across measurements).

Step 2 — Measure baseline. At a baseline substrate size S_1, measure cost per substrate operation for each operation type.

Step 3 — Scale the substrate. Increase substrate size by a factor F to $S_2 = F \times S_1$. Substrate scaling is a content-volume change, not a workload change; the substrate grows while the operation set under measurement remains the same.

Step 4 — Measure scaled. At S_2, measure cost per substrate operation for each operation type using the same protocol as at S_1.

Step 5 — Compare. Compare per-operation cost at S_1 and S_2. The criterion is whether per-operation cost is constant across the scale change or scales proportionally with operation type, not super-linearly with substrate size. A read whose cost is constant across well-formed scale changes is acceptable; a read whose cost grows quadratically with substrate size is not.

Step 6 — Repeat at multiple scale factors. Run Steps 3–5 at multiple scale factors — typically 10×, 100×, and 1000× the baseline, scaled to the deployment's projected operating range — to verify that linear behavior holds across scale ranges, not only at a single tested factor. Some cost curves appear linear across small ranges and bend at larger ones; the test is designed to catch the bend.

Step 7 — Separate substrate cost from LLM cost. Throughout the measurement, track substrate operation cost separately from LLM consultation cost. The substrate-cost measurements feed the linear-cost criterion; the LLM-cost measurements are recorded for separate analysis under the architecture's other commitments.

Step 8 — Bound variance. Verify that cost variance across repeated measurements at the same substrate size is documentable as bounded non-determinism — scheduling jitter, network variance, warm-versus-cold cache differences — rather than as unbounded variance whose magnitude itself grows with substrate size.

The test is a deployment-verification procedure, not a benchmark. It uses the deployment's actual substrate operations against the deployment's actual content; the scale factors are chosen to bracket the range over which the deployment will be operated.

5. Test outputs

The test produces a pass-or-fail verdict and the underlying measurement set.

Pass conditions. All of the following hold: per-operation cost for each substrate operation type is constant across scale factors, or scales proportionally with operation type rather than super-linearly with substrate size; substrate cost is separable from LLM consultation cost, and the separation holds across scale factors; cost variance across repeated measurements at fixed substrate size is documentable as bounded non-determinism within the architecture's permitted categories; the scale-test holds across the chosen scale factor range without bending or breaking.

Fail conditions. Any of the following obtain: per-operation cost grows super-linearly — quadratically, cubically, or exponentially — with substrate size; substrate cost is not separable from LLM consultation cost, and what the deployment reports as substrate cost includes super-linear LLM context cost; cost variance at fixed substrate size is unbounded, or grows in magnitude with substrate size in a way not captured by any permitted bounded-non-determinism category; cost exhibits scale-dependent cliffs at specific substrate sizes — discontinuous jumps that cannot be characterized as linear scaling with operation type.

A failed test is diagnostic, not just verdict-bearing: which condition fails, and at which scale factor, points toward the architectural cause and the remediation path.

6. Anti-patterns the test specifically detects

Eight anti-patterns recur across deployments that approximate CKS without satisfying the linear-cost commitment in operation. Each is named below with the test condition it violates.

(a) Super-linear cost growth. Per-operation cost grows quadratically, cubically, or exponentially with substrate size, typically because the operation requires touching substrate content beyond its task-scoped subset. A query that scans the full substrate for every call is the canonical instance.

(b) LLM-cost-as-substrate-cost. The deployment's cost model conflates LLM consultation cost with substrate operation cost, so what is reported as substrate cost includes the LLM's super-linear context-cost component. The conflation hides super-linear behavior under a name the architecture commits to scale linearly.

(c) Pure context-window memory as substrate. The deployment uses the LLM context window as the substrate. There is, in the architectural sense, no substrate at all; “substrate” cost is LLM context cost, which scales quadratically with content via attention. The test fails on substrate-cost-versus-LLM-cost separation (no separation is possible) and on super-linear growth simultaneously. This is the canonical violation the test is constructed to surface.

(d) Vendor-specific scaling cliffs. Cost spikes at specific substrate sizes due to vendor architecture transitions — database sharding boundaries, indexing-mode transitions, capacity-tier boundaries — that the deployment did not anticipate. Cost is linear within tiers and discontinuous between them; the test detects the cliffs by running across enough scale factors to bracket the transitions.

(e) Hidden cost spikes from background processes. Cost variance at fixed substrate size is unbounded because background processes — vendor-side housekeeping, garbage collection, indexing rebuilds — introduce cost not categorizable within any permitted bounded-non-determinism category. The test fails on the variance condition.

(f) Variable-cost operations. The same operation against the same substrate state at the same size produces inconsistent costs across measurements, in ways that exceed bounded non-determinism. The deployment's operation-level cost behavior is not a stable property of substrate state; it is a session-dependent quantity.

(g) Super-linear search cost. Query cost grows super-linearly with substrate size due to lack of indexing or scale-inappropriate architecture. The substrate is searchable in principle but, in practice, search is a full-substrate scan — an instance of (a) specific to retrieval, common enough to name separately.

(h) Vendor-specific caching variance. Cost depends on vendor cache state in ways not bounded to permitted non-determinism categories. The deployment's cost is governed by an external state the deployment does not manage and cannot reason about.

The eight anti-patterns are not exhaustive of all possible cost-scaling failures, but they are the ones the test is specifically constructed to surface, and naming them is what allows a failed test to point at a specific remediation path rather than at a generic “cost is too high” verdict.

7. Integration with deployment verification

The test is run at five points in the deployment lifecycle.

Initial deployment validation. Before activation for operational use, scale-test at multiple sizes to establish that the cost profile holds across the planned range. The result is a baseline cost-scaling characterization that subsequent runs are compared against.

Capacity planning. Use scale-test results to predict per-operation cost at projected deployment sizes. Because the test holds across multiple scale factors, the prediction is interpolation within a verified range rather than extrapolation from a single measurement.

Performance regression detection. After architectural changes — schema changes, indexing changes, vendor configuration changes — rerun the scale-test to detect whether the cost profile has regressed. A change that is functionally correct but produces super-linear cost growth at a previously-linear operation is the kind of regression the test catches.

Vendor migration verification. When the deployment migrates between hosts that satisfy the substrate's tool-agnosticism requirements, rerun the scale-test on the new host. The migration test (specified separately, in the prior note in this cluster) verifies tool-agnosticism preservation; the linear-cost-scaling test verifies that the new host exhibits the cost-scaling profile the commitment requires. A migration can pass one and fail the other, with different remediation paths.

Pre-scale verification. Before scaling the deployment to a new substrate size — onboarding new content, expanding to new domains, growing into new participant cohorts — run a scale-test at the projected new size to verify the cost profile holds before the scale event itself.

The five points share a common shape: the test is run when something about the deployment has changed and the cost-scaling profile needs to be re-verified, rather than only at a single qualifying event.

8. Limits of the test

The linear-cost-scaling test verifies the cost-scaling characteristic the architecture commits to. It does not verify:

- **Tool-agnosticism preservation across migration.** Verified by the tool-agnosticism-migration test (the prior note in the Phase A5 cluster). The two tests are tied architecturally — vendor-independence is meaningful only insofar as cost-scaling is preserved across the tool-agnostic interface — but the verification procedures are distinct.

- **Substrate determinism beyond cost variance.** Read determinism, write determinism, and the broader determinism contract are verified by separate tests. The cost-scaling test verifies only that variance in *cost* across measurements is bounded; it does not verify variance in *content* or in *behavior*.
- **LLM operational cost.** LLM cost is bounded within cells per a separate architectural commitment; the linear-cost-scaling test specifically excludes it from the substrate-scaling target.
- **Absolute cost adequacy.** Whether the per-operation cost is "low enough" for the deployment's economic context is a business question, not an architectural one. The test verifies the curve shape; it does not verify the level.
- **Business cost-effectiveness.** Whether the deployment is cost-effective for its intended use depends on usage patterns, alternative architectures, and the value the deployment produces. The test does not weigh in.

A deployment that passes the linear-cost-scaling criterion but fails an adjacent test is not CKS-coherent in the broader sense; a deployment that passes all the substrate-operational tests but fails the business cost-effectiveness evaluation is CKS-coherent and economically inappropriate. The two are different judgments, and bundling them produces neither verification well.

9. The test in one sentence

The CKS linear-cost-scaling test passes if and only if, across multiple substrate-size scale factors, per-substrate-operation cost is constant or scales proportionally with operation type rather than super-linearly with substrate size; substrate cost is held separable from LLM consultation cost throughout the measurement; cost variance at fixed substrate size is documentable within the architecture's permitted bounded-non-determinism categories; and no scale-dependent cliffs, hidden cost spikes, variable-cost operations, or vendor-specific caching variances appear in the measurement set.

10. Why naming the test as standalone matters

Conflating the linear-cost-scaling test with the broader determinism, tool-agnosticism, or migration tests makes verification harder than it needs to be: a deployment that fails on cost-scaling is reported as failing a bundled test, and the failure attributes ambiguously across the bundle's components. Naming the test as standalone separates the verdict from the bundle and gives operators a precise diagnosis when verification fails. It also makes the test reusable across the deployment lifecycle: the same procedure runs at initial validation, capacity planning, regression detection, migration verification, and pre-scale verification, with the verdict comparable across runs because the procedure is the same.

The standalone framing continues the substrate-operational-properties cluster the prior tool-agnosticism-migration test initiated. The next note in the cluster formalizes the conflict-coexistence test as standalone procedure; subsequent notes cover composition-requirements tests, pattern-mapping, and substrate-reproducibility. The cluster's shared shape — test as architectural

procedure, with the architectural commitment, the procedure, the outputs, the anti-patterns, the integration, and the limits each named separately — makes the cluster citable as a unit and each test usable independently.

Subsequent work that implements, extends, or argues against the CKS linear-cost-scaling commitment should use the test in the form formalized here. Work that uses the term differently — most often by failing to separate substrate cost from LLM cost, or by reporting a single-scale measurement and calling it a scaling test — is using a different procedure, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

Companion derivation notes

Li, W. (2026). *Linear-Cost Scaling: What Grows Linearly in CKS, What Doesn't, and Why.* 25 April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *The Determinism Contract: What CKS Substrates Must Guarantee About Reproducibility, and What Breaks It.* 24 April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *Tool-Agnosticism: The Three Minimal Requirements for a CKS Substrate Host.* 1 May 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *The Linear-Cost-Scaling Test as Standalone Operational Procedure in the Coordination Knowledge Substrate Pattern.* May 7, 2026. ORCID: 0009-0004-8065-3235.